```
      fmt.Println(i, "even")
    } else {
      fmt.Println(i, "odd")
    }
  }
}
```

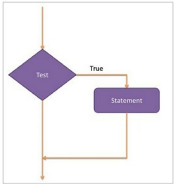

Figure 2: The *if* structure

Loops

When one needs to run the same code multiple times, loops come into play. These enable the program to execute the statements several times. The basic structure for loops is shown in Figure 3.

Here is the sample program to print numbers from 1 to 50.

```
package main
import "fmt"
func main() {
  i := 1
  for i <= 50 {
    fmt.Println(i)
    i = i + 1
  }
}
```

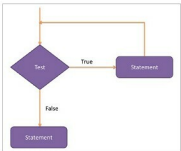

Figure 3: Loop structure

Arrays and structures

Arrays: An array is a fixed-size sequential set of data items of the same type. To declare an array of some datatype, use the following syntax:

```
 var array_name [SIZE] data_type
//declaration of an array
  var y [5]int
// y is an array of 5 integer data items.
```

Initialisation of an array can be done using a single statement as follows:

```
var salary = [5]float32{10000, 20000, 3400,70000,5000}
```

Here, the number of elements can't be more than the size of the array. If you omit the size of the array, an array can hold as many elements as you can include. Here is the sample code that describes the use of an array in Go language:

```
 package main
 import "fmt"
 func main() {
   var n [5]int
/* n is an array of 10 integers */
   var i,j int
      for i = 0; i < 5; i++ {
      n[i] = 10*i;
// set value of an element at location i to 10*i
   }
   for j = 0; j < 10; j++ {
      fmt.Printf("Value at Element[%d] = %d\n", j, n[j] )
   }
 }
 //Output
 Value at Element [0] = 0
 Value at Element [1] = 10
 Value at Element [2] = 20
 Value at Element [3] = 30
 Value at Element [4] = 40
```

Structures: A structure is a user defined data type, which enables us to make a set of data items of different types.

Structure definition and declaration syntax is as follows:

```
type struct_name struct {
   member member_type;
   member member_type;
   ...}
//Structure definition
var_name := struct_name {value1, value2...valuen}
//declaration of structure variable
```

The member access operator (.) is used to access a member. This operator is coded between the structure variable name and the structure member variable that we want to access.

```
type Employee struct {
   ID int;
   Name string;
   Salary int;
   Department string;
}
//Employee structure definition
/* Declaration of emp1 of type Employee */
var emp1 Employee
/* emp1 specification */
emp1.ID = 822141;
emp1. Name = "Palak";
emp1. Salary = 50000;
emp1. Department = "Oracle";
emp1.ID = 822141;
```

Interfaces

Interfaces are nothing but a set of method declarations. To declare the interface, use the following syntax:

```
 type interface_name interface {
             method_name1 [return_type]
             method_name2 [return_type]..}
```

The *struct* data type is used to implement the interfaces to